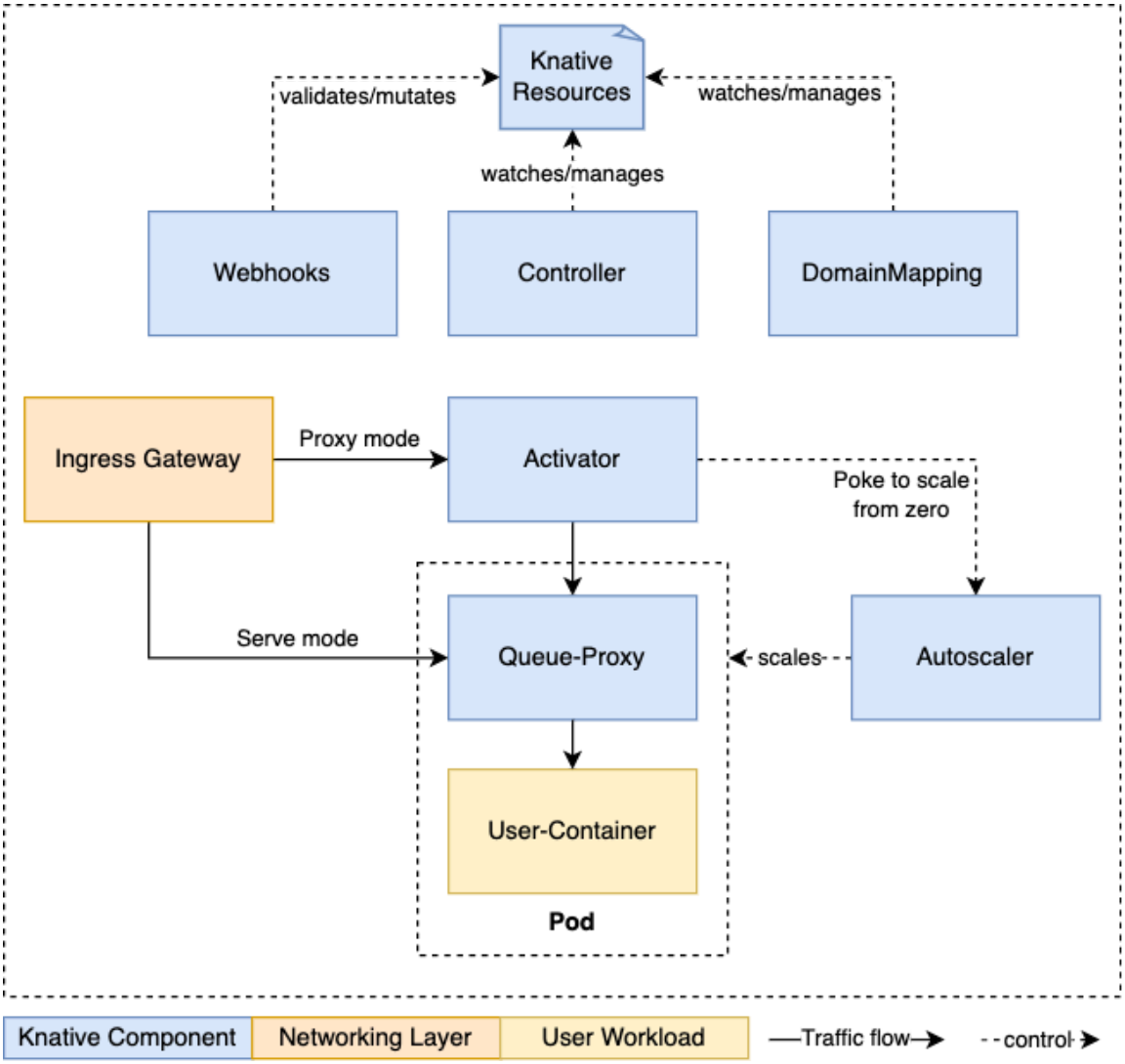
Knative Serving Architecture

Knative Serving consists of several components forming the backbone of the Serverless Platform. This page explains the high-level architecture of Knative Serving. Please also refer to **the Knative Serving Overview** and **the Request Flow** for additional information.

Diagram



Components

Component	Responsibilities
Activator	The activator is part of the **data-plane**. It is responsible to queue incoming requests (if a `Knative Service` is scaled-to-zero). It communicates with the autoscaler to bring scaled-to-zero Services back up and forward the queued requests. Activator can also act as a request buffer to handle traffic bursts. Additional details can be found **here**.
Autoscaler	The autoscaler is responsible to scale the Knative Services based on configuration, metrics and incoming requests.
Controller	The controller manages the state of Knative resources within the cluster. It watches several objects, manages the lifecycle of dependent resources, and updates the resource state.
Queue-Proxy	The Queue-Proxy is a sidecar container in the Knative Service's Pod. It is responsible to collect metrics and enforcing the desired concurrency when forwarding requests to the user's container. It can also act as a queue if necessary, similar to the Activator.
Webhooks	Knative Serving has several webhooks responsible to validate and mutate Knative Resources.

Networking Layer and Ingress

> **Note**
>
> `Ingress` in this case, does not refer to the **Kubernetes Ingress Resource**. It refers to the concept of exposing external access to a resource on the cluster.

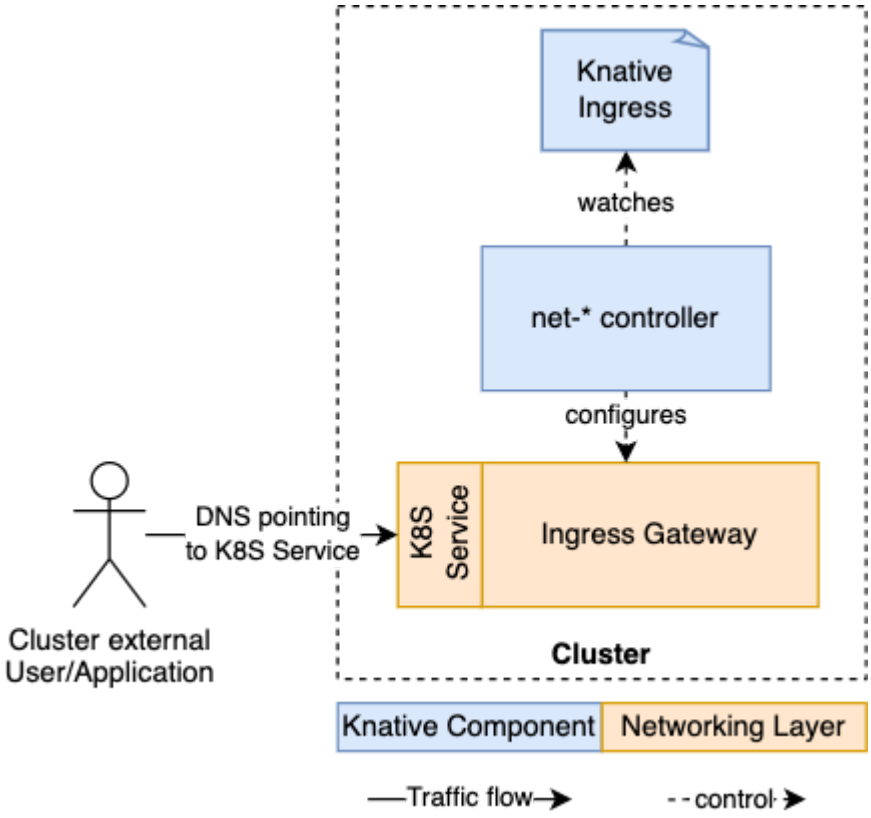
Knative Serving depends on a `Networking Layer` that fulfils the **Knative Networking Specification**. For this, Knative Serving defines an internal `KIngress` resource, which acts as an abstraction for different multiple pluggable networking layers. Currently, three networking layers are available and supported by the community:

- **net-kourier**
- **net-contour**

- net-istio

Traffic flow and DNS

> **Note**
>
> There are fine differences between the different networking layers, the following section describes the general concept. Also, there are multiple ways to expose your `Ingress Gateway` and configure DNS. Please refer the installation documentation for more information.



- Each networking layer has a controller that is responsible to watch the `KIngress` resources and configure the `Ingress Gateway` accordingly. It will also report back `status` information through this resource.
- The `Ingress Gateway` is used to route requests to the `activator` or directly to a Knative Service Pod, depending on the mode (proxy/serve, see **here** for more details). The `Ingress Gateway` is handling requests from inside the cluster and from outside the cluster.
- For the `Ingress Gateway` to be reachable outside the cluster, it must be **exposed** using a Kubernetes Service of `type: LoadBalancer` or `type: NodePort`. The community

supported networking layers include this as part of the installation. Then **DNS** is configured to point to the `IP` or `Name` of the `Ingress Gateway`

> **Note**
>
> Please note, if you do use/set DNS, you should also set the **same domain** for Knative.

Autoscaling

You can find more detailed information on our autoscaling mechanism **here**.